

# HelixSkills

## HelixSkills

适用于 CLI AI 代理的受治理、基于宪制的技能系统

### 概要

HelixSkills 是一套为 CLI AI 代理设计的技能系统，其内嵌 Helix Constitution 作为子模块，因此所有通用治理规则均无条件适用。该系统将可安装的代理技能、MCP 工具服务器、Claude Code 插件及可复用引擎整合至一个可注册、文档化的目录中。

### 简要说明

HelixSkills 是一套为 CLI AI 代理设计的技能系统。它将 Helix Constitution 作为子模块嵌入，确保所有通用规则适用，并提供可注册的技能（包括动作前缀、媒体验证、多轨处理、会话同步、可工作项生命周期等）、两个 MCP 工具服务器、两个 Claude Code 插件及可复用引擎。

### 详细说明

HelixSkills（代码库 `skills`，Apache-2.0 许可）是一套为 CLI AI 代理设计的技能系统，其设计理念刻意颠覆了常规顺序：治理优先，能力次之。它将 Helix Constitution 作为 `constitution/` 子模块引入，因此来自 `constitution/CLAUDE.md` 和 `constitution/Constitution.md` 的所有通用规则均无条件适用——这些规则并非代理可选择遵循的约定，而是物理嵌入项目树的硬性规则集。采用 HelixSkills 的代理无法规避宪制，规则与代码同行。

大多数“技能框架”仅提供抽象概念，而 HelixSkills 则直接交付一套具体、可注册的技能清单，供用户直接指向并安装。通过 `register.sh` 可安装七项宪制技能：`action-prefix-system`（动作前缀系统）、`media-validator`（媒体验证）、`multitrack`（多轨处理）、`reporting-workable-items`（可工作项报告）、`scheduled-work-queue`（定时工作队列）、`session-sync`（会话同步）及 `workable-item-lifecycle`（可工作项生命周期）——覆盖从规范化动作命名、媒体验证到工作单元完整生命周期的中高级功能。

此外，系统已预置并索引了多项草案技能（如 Android 概览、Java/Kotlin 语言、Linux 操作系统），待激活后即可使用。两个 MCP 工具服务器（media-validator、scheduled-work）通过 Model Context Protocol 向代理开放这些技能，而两个 Claude Code 插件（helix、scheduled-work）则将相同能力直接注入代理运行时——同一套技能，通过不同接口触达代理。

在目录底层，四个一级可复用引擎构成技能背后的共享机制：continuum（已实现），以及 session\_orchestrator、token\_optimizer 和 clickup\_sync（设计中），避免技能重复构建基础设施。其中，token\_optimizer 明确声明了依赖关系图，涵盖 vasic-digital 生态系统的多个组件（TOON、Embeddings、VectorDB、Normalize、conversation）及 HelixDevelopment 的 LLMProvider，确保跨代码库的连接可审计而非隐式依赖。整个系统配备严谨的文档体系：技能目录、自动生成的技能关系图索引、各代码库详情页，以及坦诚的《缺口与风险》清单，直陈尚未完成的部分。为确保韧性与区域访问便利，整套系统在 GitHub、GitLab、GitFlic 及 GitVerse 上进行了镜像部署。

## 我们为何构建它

CLI与AI代理需要的是一致、受控且可复用的能力——而非各自重新发明规则的临时脚本。HelixSkills的诞生，正是为了赋予代理一套可打包、可注册的技能集，并将其与共享的宪章绑定，确保每个采用它的代理和项目在行为上保持一致且可审计。

## 为何它是变革性的

它让代理能力在构建之初便具备可移植性与规则合规性，而非依赖于自律。每项技能都是一个受控、可版本化、可安装的单元，由宪章子模块支撑——代理注册技能的瞬间，便自动继承规范的规则集，毫无偏差空间。这解锁了此前无法实现的可能性：将一项能力从一个代理或项目迁移至另一个，确信其抵达时已绑定相同的治理框架，并通过标准化接口（MCP服务器与Claude代码插件）呈现，而非一堆各自为政的粘合脚本，每次都重新发明规则。

## 创新之处

- 宪章即子模块：通用治理规则通过继承而非复制获得——直接挂载至代码树中，确保每个使用代理都绑定同一套规范规则集，更新源自单一真实来源，而非散落在十几个过时的副本中。
- 技能以自注册单元交付（register.sh），并编织进自动生成的技能图谱索引，确保目录始终可发现，且与实际安装内容保持同步。
- 多接口暴露：同一技能集可通过MCP工具服务器及Claude代码插件触达代理——一次编写，适配代理使用的任意运行时。

- 可复用的一级深度引擎（`continuum`、`token_optimizer`、`session_orchestrator`、`clickup_sync`）在生态系统内共享，每个引擎均携带明确、可审计的跨仓库依赖声明，避免隐式耦合。

## 最大的技术挑战及解决方案

- 如何确保代理行为在众多技能与代理间保持一致且规则合规——为每项技能单独实现治理机制必然导致分歧。解决方案是将 Helix Constitution 作为子模块挂载，使 `constitution/CLAUDE.md` 与 `constitution/Constitution.md` 中的规则无条件适用，并从单一上游更新，而非被复制后逐渐过时。
- 如何让不断增长的技能集可安装且可发现——若无法找到或安装，目录便形同虚设。解决方案是通过每项技能的 `register.sh` 注册机制，在安装时自动接入技能，辅以自动生成的 INDEX 技能图谱及每仓库详细文档，确保发现机制始终与实际安装内容同步。
- 如何触达使用不同运行时的代理——同一能力不应为每个宿主重新构建。解决方案是将单一技能集同时封装为 MCP 工具服务器定义（位于 `constitution/mcp/`）与 Claude 代码插件（位于 `constitution/plugins/`），使同一实现能跨接口暴露。

内容

## 技术栈

- **Shell**（主语言）——选用该语言的原因在于，安装和注册工具必须能在任何代理运行的环境中直接执行，无需预先启动运行时。它驱动着 `register.sh` 和 `install_upstreams` 脚本，确保入门依赖零负担且高度可移植。
- **Git** 子模块——选用该方式以继承治理规则，避免重复：Helix 的 Constitution 文件以实时引用形式挂载于 `constitution/` 目录下，规则更新仅需修改一个指针，而非复制粘贴后遗忘。
- **Model Context Protocol (MCP)** ——选用其作为代理的标准工具接口，不依赖特定运行时。`constitution/mcp/` 目录下定义了两个 MCP 服务器（`media-validator` 和 `scheduled-work`），将技能以可调工具形式暴露。
- **Claude** 代码插件——选用该方式以零粘合代码将技能原生嵌入代理运行时。`constitution/plugins/` 目录下提供了两个插件（`helix` 和 `scheduled-work`），为不同宿主环境复刻了 MCP 的接口表面。
- 可复用引擎（**`continuum`、`token_optimizer`、`session_orchestrator`、`clickup_sync`**）——选用该架构将共享机制从单个技能中抽离，实现跨项目复用。例如，`token_optimizer` 通过声明式依赖而非代码复制，与 `vasic-digital` 的多个组件（`TOON`、`Embeddings`、`VectorDB`、`Normalize`、`conversation`）及 HelixDevelopment 的 `LLMProvider` 打通。
- 多宿主 **Git** 镜像（**GitHub**、**GitLab**、**GitFlic**、**GitVerse**）——选用该方案以确保单一宿主故障或区域封锁不会切断访问。同一代码库在四个平台上保持实时同步，确保韧性与可达性。

## 状态与诚信说明

- 状态：测试版。七项宪章技能、两个 MCP 服务器及两个插件已发布；草案技能已编入索引但尚未激活，四个一级深度引擎（session\_orchestrator、token\_optimizer、clickup\_sync）中有三个仍处于设计阶段。
- README 文件中将项目称为 helix\_skills，而规范的 GitHub 路径为 HelixDevelopment/skills。README 中的「已跟踪问题数」为自行报告数据。

优先级：Helix-主优先。